EEET2601 Engineering Computing 1

Strings

Quang Tran

RMIT UNIVERSITY

Character

- ASCII (American Standard Code for Information Interchange) is a 7-bit code that uses integers in the range [0, 127] to represent characters
- The ASCII code of a character can be described by either an integer or a pair of single quotes around the character

```
char ch = 65;
char ch = 'A';    // preferred
```

ASCII Table

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	|
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

<ctype.h>

Function	Description
int isalpha(int c)	Checks whether the passed character is alphabetic
int isdigit(int c)	Checks whether the passed character is decimal digit
int isalnum(int c)	Checks whether the given character is alphanumeric
int ispunct(int c)	Checks whether the passed character is a punctuation character
int isspace(int c)	Checks whether the passed character is a space
int iscntrl(int c)	Checks whether the passed character is a control character
int islower(int c)	Checks whether the passed character is lowercase letter
int isupper(int c)	Checks whether the passed character is uppercase letter
int tolower(int c)	Converts an uppercase letter to lowercase
int toupper(int c)	Converts a lowercase letter to uppercase

Character

```
// Print all ASCII characters
printf("%3s%10s%10s\n", "Dec", "Char",
                        "Type");
for (int i = 0; i <= 127; i++) {
    printf("%3d%10c", i, i);
    if (isalpha(i)) {
        printf("%10s", "letter");
    } else if (isdigit(i)) {
        printf("%10s", "digit");
    } else if (ispunct(i)) {
        printf("%10s", "punct");
    } else if (isspace(i)) {
        printf("%10s", "space");
    } else if (iscntrl(i)) {
        printf("%10s", "control");
    } else {
        printf("%10s", "other");
    }
    printf("\n");
}
```

String

- A string in C is **an array of characters** ended with the **null character '\0'**, which has the ASCII value 0

```
char s1[ ] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

- A string can also be described by a pair of double quotes around a sequence of characters without specifying the null character explicitly

```
char s2[ ] = "Hello";      // '\0' is auto inserted
```

String

```
// Two equivalent ways to declare the same string
char s1[] = {'H', 'e', 'l', 'l', 'o', '\0'};
char s2[] = "Hello";

// Print characters as decimal integers and characters
printf("%5s%5s%5s%5s\n", "Dec", "Char", "Dec", "Char");
for (int i = 0; i < 6; i++) {
    printf("%5d%5c%5d%5c\n", s1[i], s1[i], s2[i], s2[i]);
}
```

I/O functions

Function	Description
int getchar()	Read a character from stdin. Return the character or EOF (-1) on end of file or error.
int putchar(int char)	Write a given character to stdout. Return the written character or EOF on error.
int scanf(const char *format, ...)	Read formatted input from stdin. Return the number of items successfully read or a negative value for error.
int printf(const char *format, ...)	Write formatted output to stdout. Return the number of characters successfully written or a negative value for error.
Note: putchar(), scanf(), and printf() can be called without storing the return values	

I/O functions

```
// Print the value of EOF
printf("EOF is %d\n", EOF);
```

end of file : -1

```
// Read a character from stdin
printf("Enter a character: ");
int char1 = getchar();
```

```
// Write a character to stdout
int char2 = putchar(char1);
printf(" %d\n", char2);

// Read and write a string
char name[20];
printf("Enter your name: ");
scanf("%s", name);
printf("My name is %s\n", name);
```

I/O functions

- What if you want to input your full name with some spaces?

```
// Read and write string with spaces
char name[20];
printf("Enter your name: ");
scanf("%[^\n]s", name);    // what's special here?
printf("My name is %s\n", name);
```

accept everything except the newline

<string.h>

Function	Description
unsigned long strlen(const char *s)	Return the number of characters in s, up to but not including the null character '\0'.
int strcmp(const char *s1, const char *s2)	Compare s1 to s2. Return 0 if they are the same, negative or positive if s1 < s2 or s1 > s2 lexicographically.
int strncmp(const char *s1, const char *s2, size_t n)	Compares at most the first n bytes of s1 and s2. Return 0 if they are the same, negative or positive if s1 < s2 or s1 > s2 lexicographically.
char *strcpy(const char *s1, const char *s2)	Copy s2 to s1, then return s1. You must make sure that s1 has enough space to store the result.
char *strncpy(char *s1, const char *s2, size_t n)	Copy up to n characters from s2 to s1, then return s1. You must make sure that s1 has enough space to store the result.

<string.h>

Function	Description
char *strcat(const char *s1, const char *s2)	Append s2 into s1, then return s1. You must make sure that s1 has enough space to store the result.
char *strncat(char *s1, const char *s2, size_t n)	Append up to n characters from s2 into s1, then return s1. You must make sure that s1 has enough space to store the result.
char *strstr(const char *s1, const char *s2)	Finds the first occurrence of the entire string s2 (not including the terminating null character) which appears in the string s1. Return NULL if s2 is not found in s1.
char *strchr(const char *str, int c) left -> right	Searches for the first occurrence of the character c (an unsigned char) in the string str.
char *strrchr(const char *str, int c) right -> left	Searches for the last occurrence of the character c (an unsigned char) in the string str.
char *strtok(char *str, const char *delim)	Breaks string str into a series of tokens separated by delim.

delimiter -> skip "char"

<string.h>

```
// Create strings
char s1[] = "Apple";
char s2[] = "Pineapple";

// Find out string's length
printf("%s's length is %lu\n", s1,
strlen(s1));
printf("%s's length is %lu\n", s2,
strlen(s2));
```

```
// Compare two strings
printf("The result of %s compare to %s
is %d\n", s1, s2, strcmp(s1, s2));

// Copy string
char s3[8];
strcpy(s3, s1);    // Fine
printf("s3 is %s\n", s3);
strcpy(s3, s2);    // Why error?
printf("s3 is %s\n", s3);
```

Array of strings

```
// Create an array of strings
char names[3][20] = {
    "Roger Federer",
    "David Beckham",
    "Tiger Woods"
};
```

```
// Print them out
for (int i = 0; i < 3; i++) {
    printf("%s\n", names[i]);
}
```

Exercise

Get three full names and a search keyword. Find and print the full names containing the search keyword. Assume that each full name has maximum 50 characters. Here is a sample run:

```
Enter full name 1: Andrew John Smith
Enter full name 2: John Doe
Enter full name 3: Minh Van Nguyen
Enter a search keyword: oh

The matched names:
Andrew John Smith
John Doe
```

References

1. Fresh2Refresh, C Programming Tutorial, 2020.
2. StudyTonight, C Programming Language Tutorial, 2020.
3. TutorialsPoint, C Standard Library, 2020.
4. Wikipedia, ASCII Table, 2010.